CS683 PA-1

Task 2 Report

Matrix Multiplication (SGEMM)

24B1021 | 24B1015 | 24B1037 | 24B0995

Software Prefetching · SIMD · llama.cpp Integration

September 7, 2026

Contents

Setup

All measurements were taken on a 13th Gen Intel Core i7-13650HX. Every build and every run was pinned to CPU 0, a P-core, with `taskset -c 0`. L1-D size is 48KB and line size is 64B.

1 Task 2A: Software Prefetching

1.1 Implementation

`matmul_prefetch` tiles M, N, and K with block size $T = 64$. Within a tile, the K-loop is unrolled by 16 and `_mm_prefetch(..., _MM_HINT_T0)` issues a lookahead load for both operands every 16 elements, at a chosen distance ahead of use. The lookahead is bounded by the true row length K (not the current K-tile's edge), so prefetching correctly continues into the next K-tile of the same row for large distances. `_mm_prefetch` was used over `__builtin_prefetch` for direct control over the locality hint.

1.2 Speedup vs. Matrix Size

Hint fixed at `_MM_HINT_T0`, $T = 64$, distance in $\{32, 64\}$, $N \in \{128, 256, 512, 1024, 1536, 2048, 3072\}$. Speedup is vs. `matmul_naive`.

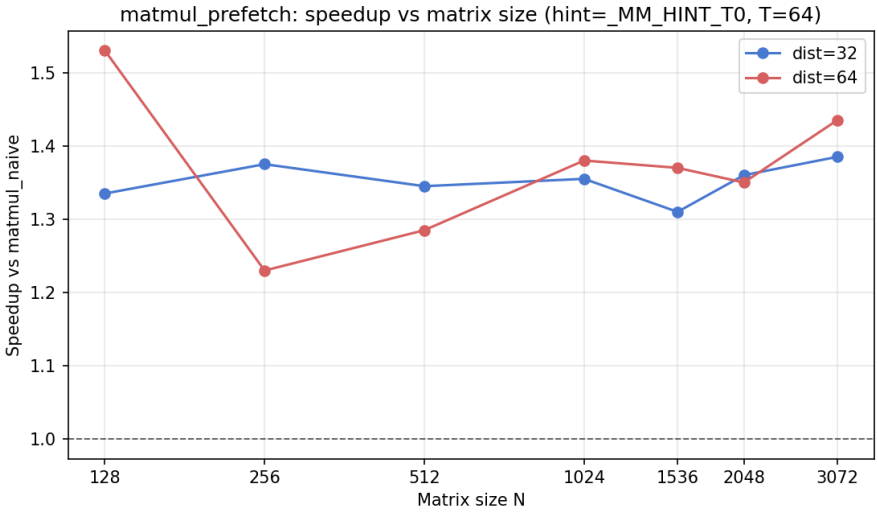

Figure 1: Speedup of `matmul_prefetch` over `matmul_naive` vs. matrix size, for distance = 32 and = 64.

N	**Speedup (dist= 32)**	**Speedup (dist= 64)**
128	1.33x	1.53x
256	1.38x	1.23x
512	1.34x	1.29x
1024	1.35x	1.38x
1536	1.31x	1.37x
2048	1.36x	1.35x
3072	1.39x	1.44x

Table 1: Speedup vs. `matmul_naive` across matrix sizes. Speedup is consistently in the 1.2–1.5× range with no strong monotonic trend; distance = 64 has the higher mean (1.37x vs. 1.35x) and wins at the largest size tested ($N = 3072$).

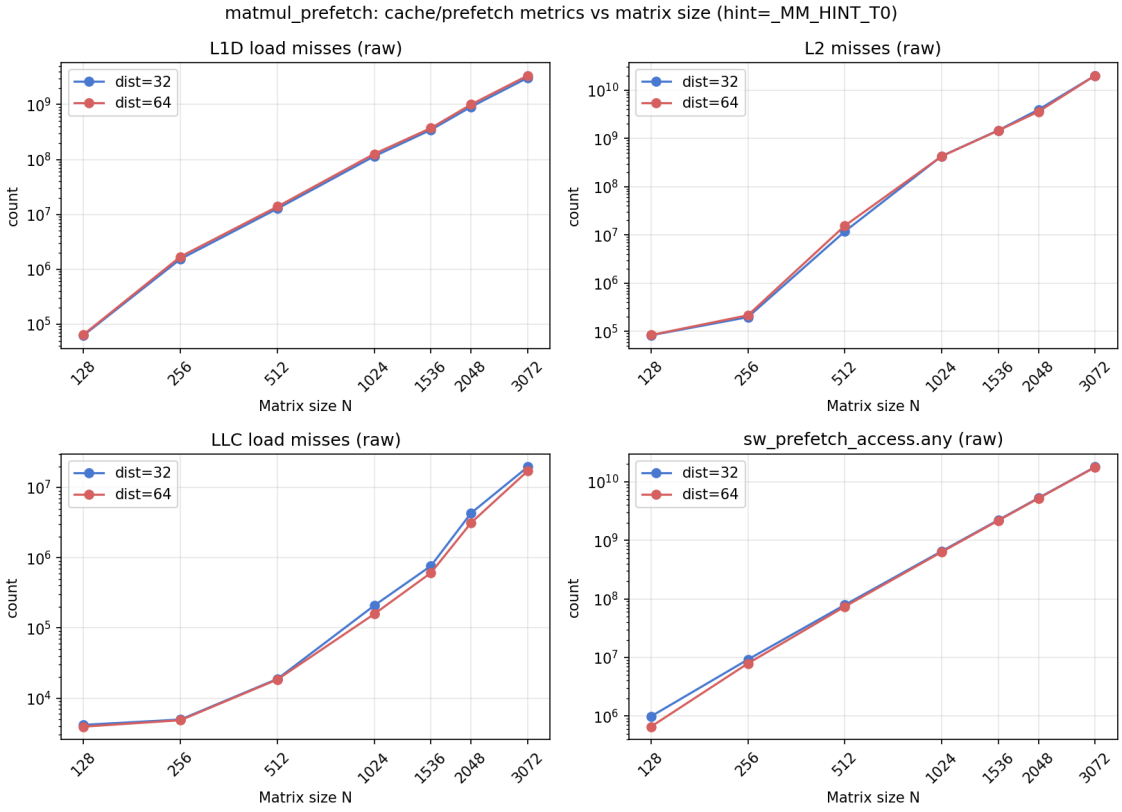

Figure 2: L1D, L2, LLC misses and `sw_prefetch_access.any` vs. matrix size (log-log), for distance = 32 and = 64. All four metrics grow monotonically with N^3-scale work; L2 misses cross into the billions and LLC misses into the tens of millions by $N = 3072$, past the point where the working set fits in LLC.

Why flat: both `matmul_naive` and `matmul_prefetch` read A and B with unit stride, so naive is already reasonably cache-friendly and not too bad at any size. Prefetching only hides a roughly constant fraction of the memory stall time behind compute at every N. It does not change what fraction of the growing miss counts in Fig. 2 are avoidable. Hence, the speedup stays in a narrow band rather than growing with N.

1.3 Speedup vs. Prefetch Distance

Hint fixed at `_MM_HINT_T0`, $T = 64$, distance swept in $\{8, 16, 32, 64, 128, 256\}$ floats at $N = 1024, 2048$. Speedup is vs. `matmul_naive` on the same workload/seed.

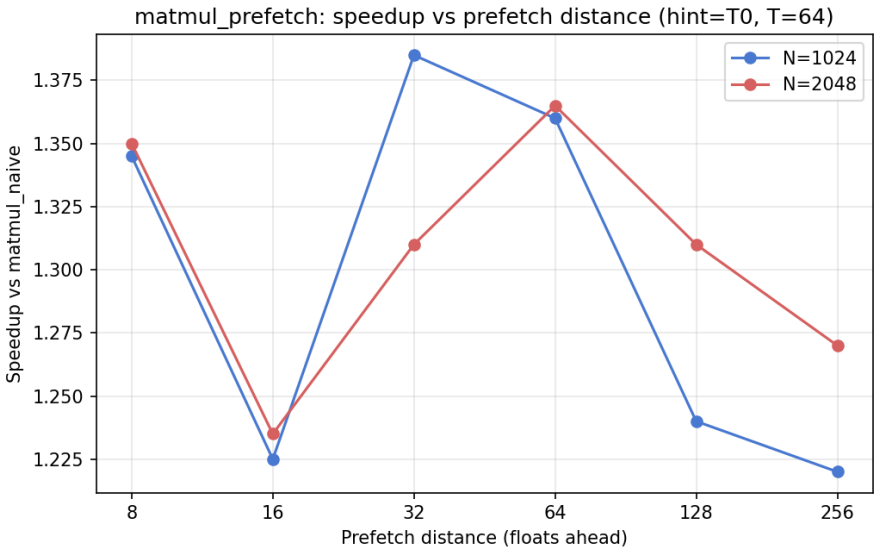

Figure 3: Speedup vs. prefetch distance for `matmul_prefetch`.

Distance (floats)	**Speedup** (N=1024)	**Speedup** (N=2048)
8	1.34x	1.35x
16	1.23x	1.23x
32	1.39x	1.31x
64	1.36x	1.37x
128	1.24x	1.31x
256	1.22x	1.27x

Table 2: Speedup vs. `matmul_naive` by prefetch distance. Best mean speedup at distance = 64 (1.36x), narrowly ahead of 32 (1.35x); 32 is actually higher at N=1024 alone, but 64 wins on the N=2048 point and the overall mean.

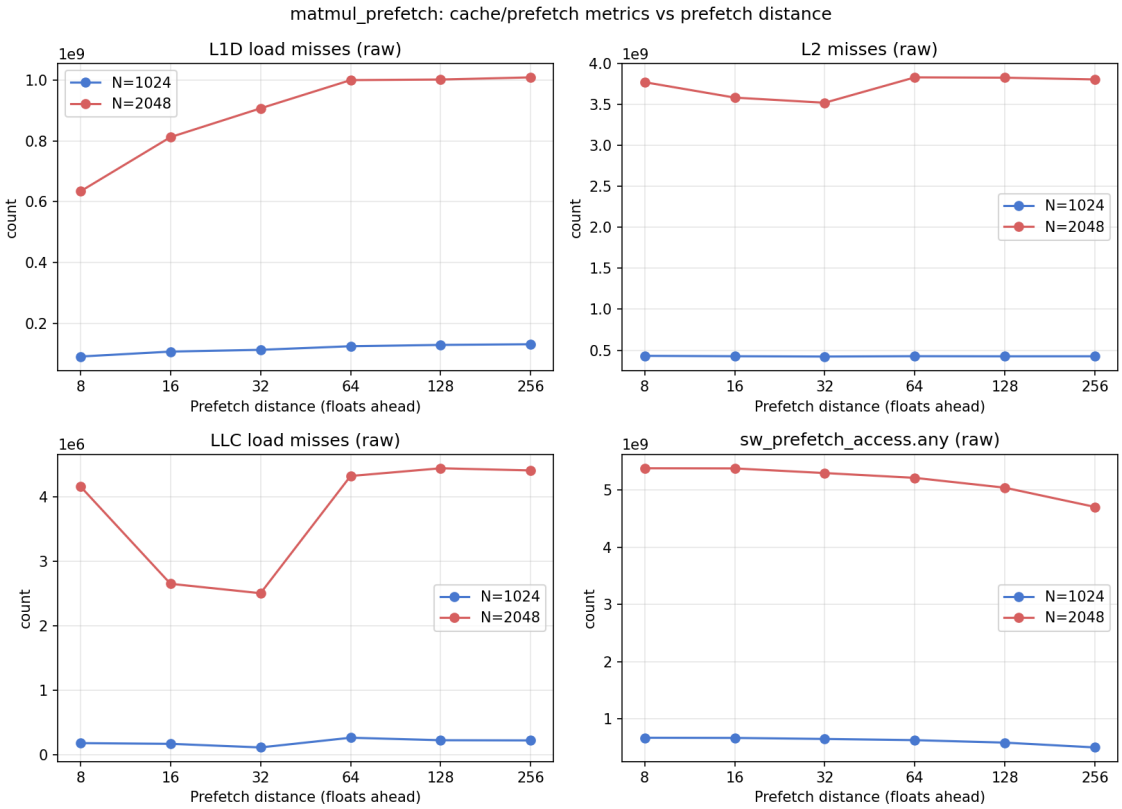

Figure 4: L1D, L2, LLC misses and `sw_prefetch_access.any` vs. prefetch distance. L1D misses and `sw_prefetch_access.any` vary smoothly and predictably with distance; L2 misses are essentially flat (within ±2%) across all six distances, and LLC misses are noisy (lowest at distance= 32, highest at 64) since they are rare events on this implementation and a median of 2 repeats does not fully average that out.

Why 64 *wins (on the mean):* it gives the load pipeline enough lead time to complete the DRAM/LLC fetch before the data is touched, without prefetching so far ahead that the line is evicted before use. 32 is close behind and even edges it out at N=1024 alone. The two sit in the same "enough but not too much" lead-time area, and the gap between them is smaller than the run-to-run noise visible in the LLC-miss counts above. Distance= 16 is the clear low point at both sizes (1.23×); it coincides with the function's K-unroll granularity of 16, so at that one distance the prefetch and its consuming load land in the same unrolled step, leaving little effective lead time.

1.4 Speedup vs. Prefetch Locality Hint

Distance fixed at 64 floats, $T = 64$, hint swept over `_MM_HINT_{T0,T1,T2,NTA}` at $N = 1024, 2048$. Speedup is vs. `matmul_naive` on the same workload/seed.

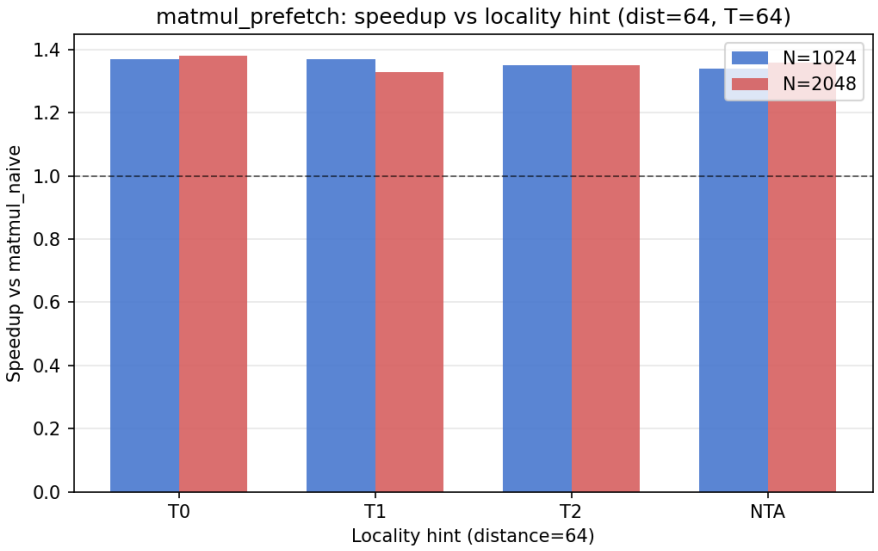

Figure 5: Speedup for `_MM_HINT_T0 / T1 / T2 / NTA` locality hints.

Hint	**Speedup** (N=1024)	**Speedup** (N=2048)
T0	1.37x	1.38x
T1	1.37x	1.33x
T2	1.35x	1.35x
NTA	1.34x	1.36x

Table 3: Speedup vs. `matmul_naive` by locality hint. Best mean speedup at `T0` (1.38x).

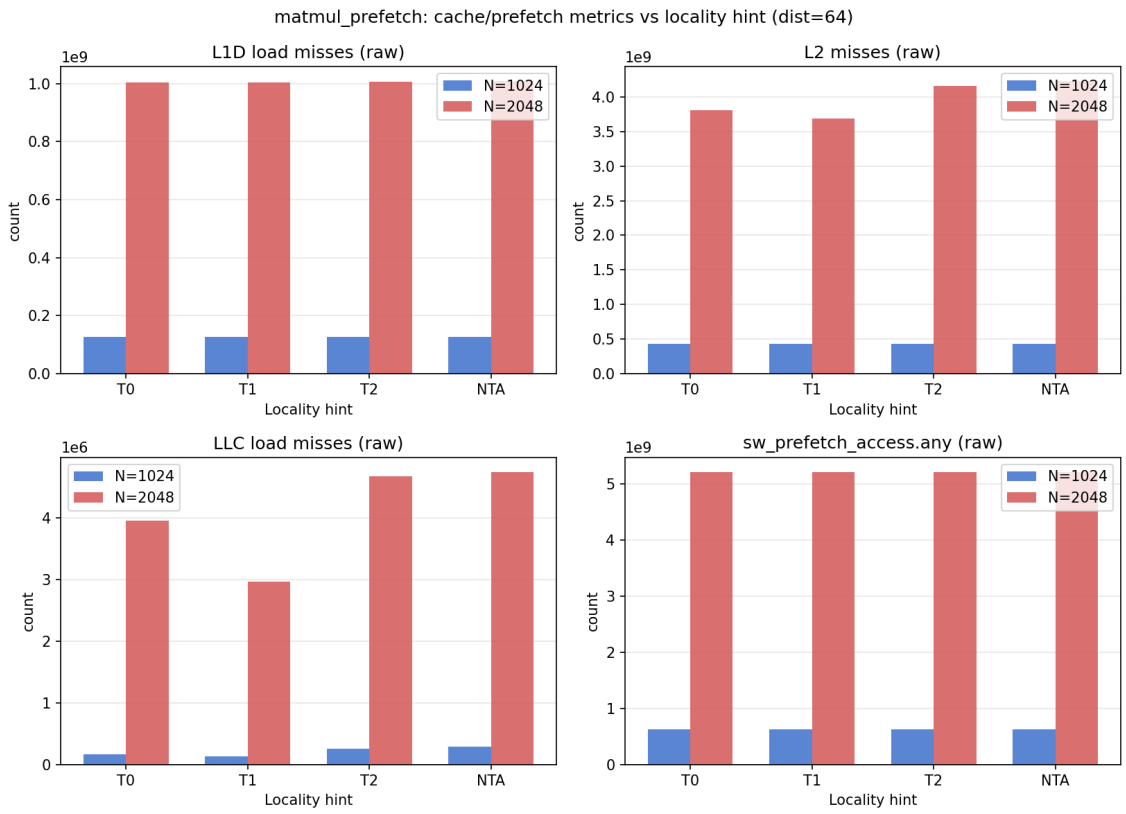

Figure 6: L1D, L2, LLC misses and `sw_prefetch_access.any` vs. locality hint. LLC misses are lowest for `T1`, highest for `NTA`; L1D misses and `sw_prefetch_access.any` are essentially flat across hints.

Why T0 (marginally) wins: with distance already near-optimal, the hint only decides which cache levels the fetched line lands in, not whether it arrives in time. `T0` fills every level including L1D, so the line is already at the point of use when reused; `T1`/`T2`/`NTA` place it in L2, LLC, or a non-temporal buffer instead, adding a small extra hop up to L1D on the actual load. That hop is cheap, so the spread between hints (1.34–1.38×) is much smaller than the spread across distances.

1.5 Effect of Hardware Prefetchers

All four L2/L3 hardware prefetchers were disabled via MSR `0x1A4` (`wrmsr -p 0 0x1a4 0xf`, restored to `0x0` after) at the best config (distance= 64, hint=T0, $T = 64$).

N	HW prefetch	naive (ms)	prefetch (ms)	speedup
1024	ON	476.3	348.0	1.37x
1024	OFF	476.7	383.7	1.24x
2048	ON	4157.1	3069.0	1.35x
2048	OFF	4930.7	3367.6	1.46x

Table 4: Effect of disabling all four HW prefetchers. At $N = 1024$, both kernels are barely affected. At $N = 2048$, `naive` slows down by 19–44% (a repeat run without `perf` overhead measured naive OFF at 5964.1ms, i.e. up to 44% slower, showing high run-to-run variance once HW prefetch is off) while `prefetch` slows by only $\sim$10% – so the naive/prefetch speedup ratio *increases* when HW prefetch is disabled.

Metric	naive ON	naive OFF	prefetch ON	prefetch OFF
L1D miss	135,566,760	2,407,962,620	1,003,615,281	5,065,277,174
L2 miss	3,743,418,813	3,281,496,014	4,188,874,265	3,376,038,747
LLC miss	2,937,937	731,057,910	5,025,087	729,032,068
sw_prefetch	136,559	160,125	5,209,576,955	5,208,353,385

Table 5: Cache metrics at $N = 2048$, HW prefetch ON vs. OFF. LLC misses jump $\sim$250$\times$ for both kernels when HW prefetch is disabled (2.9M$\rightarrow$731M naive, 5M$\rightarrow$729M prefetch) – with the streaming prefetcher gone, far more L2 misses actually reach the LLC/DRAM instead of being hidden. `sw_prefetch_access.any` is unchanged (software prefetch issue rate does not depend on the HW prefetcher setting).

1.6 Questions

1. **Trend in speedup with matrix size:** Roughly flat, 1.2–1.5$\times$ from $N = 128$ to 3072, with no strong monotonic trend (Table 1). This is because `matmul_naive` is already unit-stride on both operands, so it is never that slow to begin with. Prefetching only removes a roughly constant fraction of stall cycles at every size, rather something that scales uniformly with N. The raw miss counts in Fig. 2 grow by orders of magnitude with N, but that growth is common to both kernels, so the *ratio* stays flat even though the *absolute* misses avoided keep growing.

2. **Best prefetch distance:** 64 floats, mean speedup 1.36$\times$ across N=1024, 2048 (Table 2), narrowly ahead of 32 (1.35$\times$) and clearly ahead of 128/256 (1.27$\times$/1.25$\times$). 64 and 32 floats both give the load pipeline enough lead time to complete the DRAM/LLC fetch before the data is touched, and sit close enough together that which one wins depends on the size (32 actually edges out 64 at N=1024 alone). Distance 16 is the clear low point at both sizes (1.23$\times$); it coincides with the function's K-unroll granularity of 16, so the prefetch and the load it is meant to hide land in the same unrolled step, leaving little effective lead time. 128/256 are consistently worse than 32/64, consistent with prefetching far enough ahead that the line risks being evicted again before use.

3. **Cache fill level (locality hint) giving maximum speedup:** `_MM_HINT_T0` (fill all cache levels), mean speedup 1.38$\times$ across N=1024, 2048; `T1`, `T2`, `NTA` all 1.35$\times$ (Table 3). With distance already tuned, the hint only decides which levels the fetched line lands in, not whether it arrives in time: `T0` places it directly in L1D, the level it will actually be read from,

while `T1/T2/NTA` land it in L2, LLC, or a non-temporal buffer and pay a small extra hop up to L1D on the real load. That hop is cheap relative to a full memory fetch, so the spread across hints (1.34–1.38×) is much narrower than the spread across distances (1.23–1.36×).

4. **L1D / L2 / LLC miss trends across sizes and parameters:** All three miss counts grow monotonically with N (cubic work growth), from tens of thousands at $N = 128$ to billions (L2) / tens of millions (LLC) at $N = 3072$. Across distance and hint, L1D and `sw_prefetch_access.any` counts are nearly identical (prefetching does not change the number of loads issued, only when the data for them arrives); L2 misses are essentially flat across both sweeps, while LLC misses are the noisiest metric and do not track speedup cleanly (lowest at distance= 32, highest at distance= 64, in the distance sweep; lowest at hint=T1 in the hint sweep). LLC-miss counts alone are not a reliable predictor of which parameter setting wins.

5. **Effect of hardware prefetchers:** Disabling them hurts `naive` far more than `matmul_prefetch` at $N = 2048$ (19–44% slower vs. ∼10% slower), so the naive/prefetch speedup ratio goes *up* when HW prefetch is off (1.35× → 1.46×). This is the clearest confirmation that software prefetch and the hardware prefetcher are substitutes for the same job (hiding memory latency): `matmul_prefetch` has its own mechanism to fall back on when the hardware one is removed, while `naive` has nothing, so it absorbs almost the full cost. LLC misses jump ∼ 250× for both kernels when HW prefetch is disabled (2.9M→731M naive, 5M→729M prefetch), confirming the hardware prefetcher was previously intercepting the large majority of L2 misses before they reached the LLC/DRAM.

2 Task 2B: SIMD (AVX2)

2.1 Implementation

The function keeps the natural $i \to j \to p$ loop order but register-blocks the j (column-of-B) loop by `step` = 4: each iteration of the inner loop computes four output elements $C[i][j\,..\,j{+}3]$ at once. Four `__m256` accumulators (`acc0..acc3`) are held in registers and zeroed at the top of the block. The p loop then advances 8 floats at a time: one `_mm256_loadu_ps` pulls a lane of $A[i][p\,..\,p{+}7]$, which is reused across all four columns, and four `_mm256_fmadd_ps` multiply it by the four B-row lanes and add into the accumulators. This 1×4 register block amortizes the single A load and the loop overhead over four FMAs, and the four independent accumulator chains hide the FMA latency.

After the vector loop, each accumulator is reduced to a scalar by `hsum256` (an `extractf128` + `add` to fold the two 128-bit halves, then two `hadd_ps`). Remainders are handled by scalar cleanup: a `p < K` tail loop finishes the last K mod 8 elements into the same four sums, and a `j < N` tail loop handles the last N mod 4 columns one at a time via the scalar-tail helper `dot_simd`. All loads are unaligned (`loadu`) so arbitrary `lda/ldb/ldc` are safe.

2.2 Baseline Profiling / Instruction Count

User-space instructions were attributed to each function with `perf record -e instructions:u` followed by `perf report`: the per-call figure is the run's total event count times the function's overhead percentage, divided by the number of times the main function calls that function in a single-stage run (naive: 1 reference + 1 warmup + 1 + 3 timed = 6; SIMD: 1 warmup + 1 + 3 = 5). Table 6 gives the 1024^3 point; Fig. 7 shows the full size sweep.

The reduction is essentially size-independent, ≈ 11.8× for the 128-bit implementation and ≈ 23× for the 256-bit implementation across the whole sweep, because it comes from the code's

structure, not from cache behaviour. It exceeds the raw lane counts (4 and 8) because each vector iteration also folds a multiply and an add into one FMA and amortizes one A load, the loop-counter arithmetic and the branch over four columns of output. Doubling the vector width from 128 to 256 bits roughly halves the count, as expected.

Stage	Instr./call (1024^3)	Reduction vs naive
`matmul_naive`	6.45×10^9	1.00×
`matmul_simd` (128-bit)	5.45×10^8	11.8×
`matmul_simd` (256-bit)	2.79×10^8	23.1×

Table 6: User-space instructions per call at M=N=K=1024, from `perf record`/`perf report`.

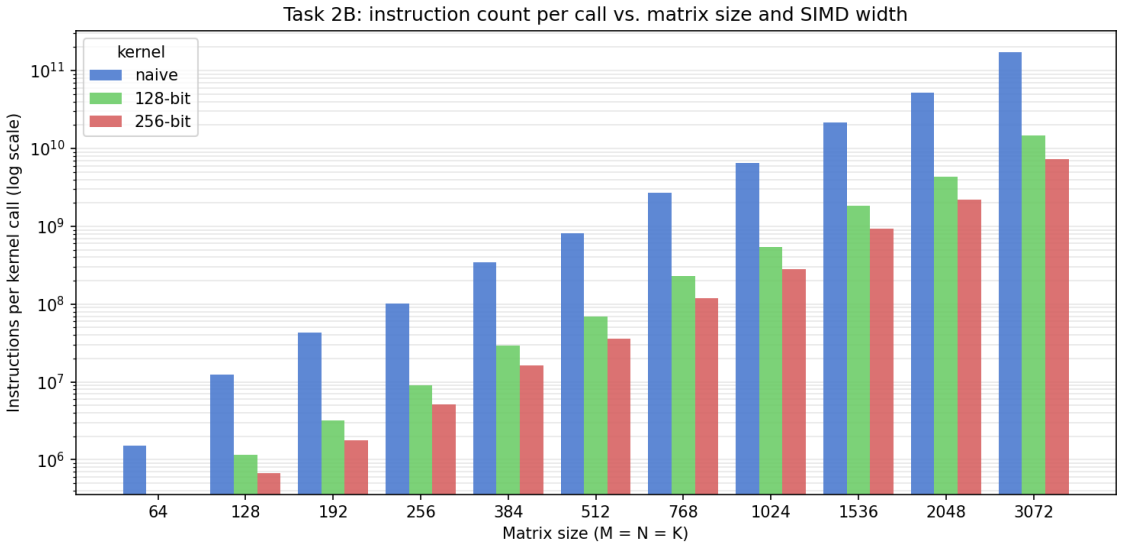

Figure 7: Instructions executed per call vs. matrix size (M=N=K), for `matmul_naive`, the 128-bit SIMD, and the 256-bit implementations. Per-call figures are the per-symbol instruction count from `perf record` (`event_count` × overhead %) divided by the number of times the main function calls each function in a single-stage run (naive: 6, SIMD: 5). Log scale.

The 128-bit and 256-bit SIMD bars are absent at N=64: that workload runs in a few microseconds and `perf` collected too few samples so those are ditched here.

2.3 Performance

Stage	Time (ms)	GFLOP/s	Speedup
`matmul_naive` (ref)	496.506	4.33	1.00×
`matmul_simd` (256-bit)	92.051	23.33	5.39×

Table 7: `./bin/matmul simd 1024 1024 1024 1234`. At smaller, cache-resident sizes the SIMD speedup is much higher (up to 19× at N=256); see Fig. 8.

5.39× is well below the 8× raw lane count of SIMD because at N=1024 the $N \times K$ float working set no longer fits in L2/L3: the kernel is already partly memory-bound here, so the

23× instruction-count reduction (Table 6) does not translate one-to-one into performance time speedup.

2.4 Speedup vs. SIMD Width

The final `src/matmul_simd.cpp` is the 256-bit SIMD implementation. Both implementations were swept over $M{=}N{=}K \in \{64, \ldots, 3072\}$ against `matmul_naive`; the speedups are plotted in Fig. 8 and the absolute runtimes in Fig. 9. Below $N \approx 512$ the problem is cache-resident and the wider register roughly doubles the speedup (19× vs 8× at $N{=}256$); beyond the L2/L3 capacity both kernels become memory-bound and converge to ≈ 4–5×, where vector width no longer matters.

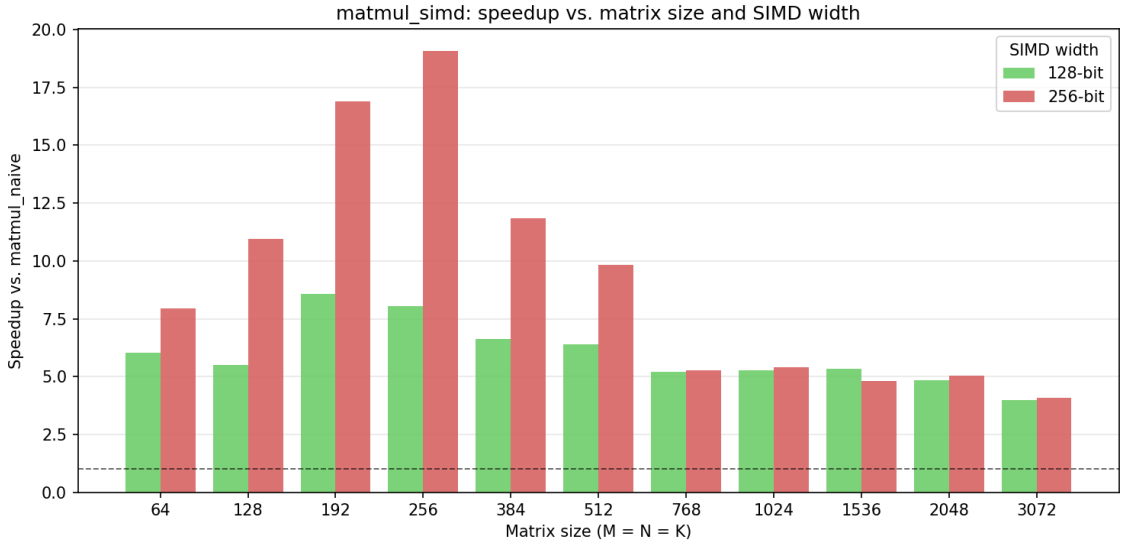

Figure 8: Speedup of `matmul_simd` over `matmul_naive` vs. matrix size ($M{=}N{=}K$), for the 128-bit and 256-bit SIMD kernels.

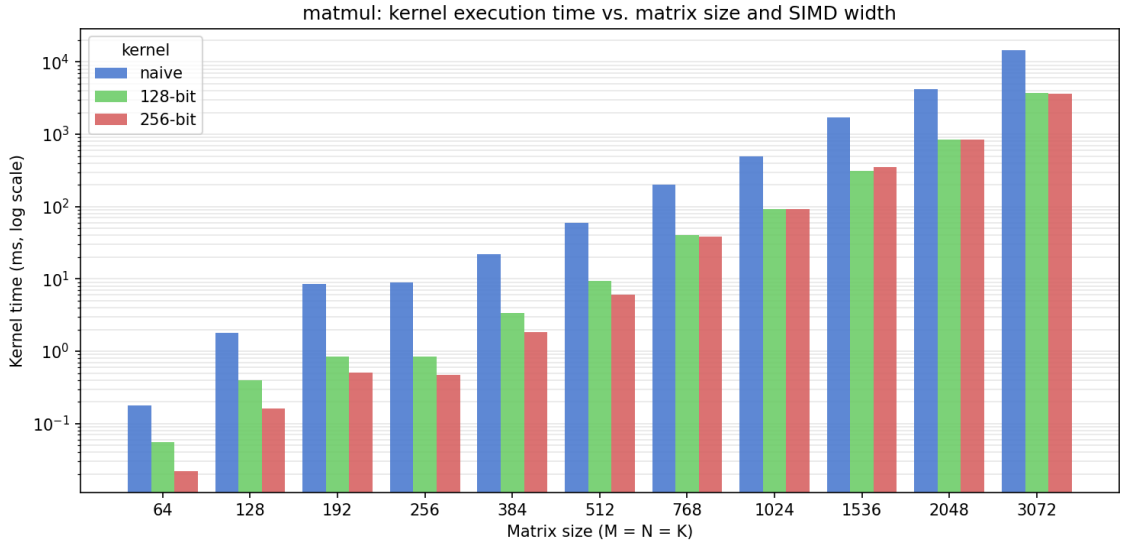

Figure 9: execution time (ms, log scale) vs. matrix size ($M{=}N{=}K$). Same runs as Fig. 8.

SIMD width	Metric	Matrix size N (M=N=K) 64	128	192	256	384	512	768	1024	1536	2048	3072
No SIMD (`matmul_naive`)	Instructions/call	1.52×10^6	1.25×10^7	4.31×10^7	1.02×10^8	3.41×10^8	8.08×10^8	2.72×10^9	6.45×10^9	2.18×10^{10}	5.16×10^{10}	1.74×10^{11}
	Time (ms)	0.177	1.778	8.598	8.934	21.990	59.789	201.384	496.506	1707.046	4249.303	14687.692
128-bit (SSE)	Instructions/call	—	1.15×10^6	3.20×10^6	9.07×10^6	2.92×10^7	6.93×10^7	2.31×10^8	5.45×10^8	1.83×10^9	4.32×10^9	1.46×10^{10}
	Time (ms)	0.056	0.395	0.844	0.843	3.362	9.502	40.829	93.164	312.891	854.104	3699.584
	Speedup vs. No SIMD	6.03×	5.51×	8.57×	8.04×	6.61×	6.38×	5.20×	5.26×	5.33×	4.84×	3.98×
256-bit (AVX2)	Instructions/call	—	6.73×10^5	1.77×10^6	5.17×10^6	1.62×10^7	3.61×10^7	1.19×10^8	2.79×10^8	9.30×10^8	2.19×10^9	7.34×10^9
	Time (ms)	0.022	0.162	0.510	0.468	1.858	6.075	38.203	92.051	354.612	843.911	3590.953
	Speedup vs. No SIMD	7.95×	10.94×	16.88×	19.08×	11.84×	9.84×	5.27×	5.39×	4.81×	5.04×	4.09×

Table 8: Instructions/call, execution time, and speedup vs. matrix size, for each SIMD width. Instructions/call are retired user-space instructions from `perf record`/`perf report` (per-call = event count × overhead% divided by function call count); times are the code's median function-only `time(ms)`, not whole process `perf stat` time. Same source data as Table 6, Fig. 7, Fig. 8, and Fig. 9.

2.5 Questions

1. **Instruction count change (naive → SIMD):** Instructions per function call drop by about 11.8× for the 128-bit implementation and 23× for the 256-bit implementation, roughly constant across matrix size (Table 6, Fig. 7). This exceeds the raw lane counts (4 and 8) for two reasons: each `fmadd` replaces a separate multiply and add, and the 1×4 register block issues one A load plus one set of loop-counter/branch instructions per four FMAs instead of per one. The count halves cleanly from 128 to 256 bits because the work per instruction doubles while the loop structure is identical.

2. **Trends across matrix size × SIMD width; which width gives max speedup:** For cache-resident sizes ($N \lesssim 512$) speedup rises with N and scales with vector width, peaking at N=256: 19.1× for 256-bit and 8.0× for 128-bit. Here the code is compute-bound, so halving the instruction count with a wider register directly halves the runtime. As N grows past the L2/L3 capacity the $N \times K$ floats no longer fits and the function streams it from memory on every row of A; both widths become memory-bound and converge to $\approx$ 4–5×, where SIMD width stops mattering. So 256-bit gives the maximum speedup at every size, but its advantage over 128-bit shrinks from $\sim 2\times$ to negligible as the workload leaves cache.

3 Task 2C: Software Prefetching + SIMD (Combined / Optimized)

3.1 Implementation

`matmul_optimized` tiles N into blocks of $T = 96$ columns of B. Within a tile it uses 4×3 SIMD "array" (4 rows of A × 3 columns of B, 12 `__m256` accumulators) with two manually-unrolled K-steps of 8 floats each (16 floats/iteration) before a scalar tail. Software prefetch (`_MM_HINT_T0`, distance 64 floats, same as Task 2A) is issued on the 3 active B rows only, since B is re-streamed once per internal loop iteration while the 4 A rows are reused across all j in a tile and mostly stay resident. Arbitrary `M/N/K/lda/ldb/ldc` are handled with scalar fallback loops on every edge: a 1-row tail when $M \bmod 4 \neq 0$, a 1-column tail when a T-tile's width isn't a multiple of 3, and a scalar (with SIMD) dot-product tail on $K \bmod 8$. Block size and unroll depth were tuned by sweeping T on N=1024, 2048. $T = 96$ (a multiple of the 4×3 tile) and the 4×3 shape were chosen because they exactly saturate the 16 available SIMD registers with no spilling: the inner loop keeps 12 accumulators + 3 live B vectors + 1 reused A vector = 16 registers live at once.

3.2 Final Performance Summary

$N \in \{128, 256, 512, 1024, 1536, 2048, 3072\}$, seed 1234. Speedup is each stage vs. `matmul_naive` on the same workload.

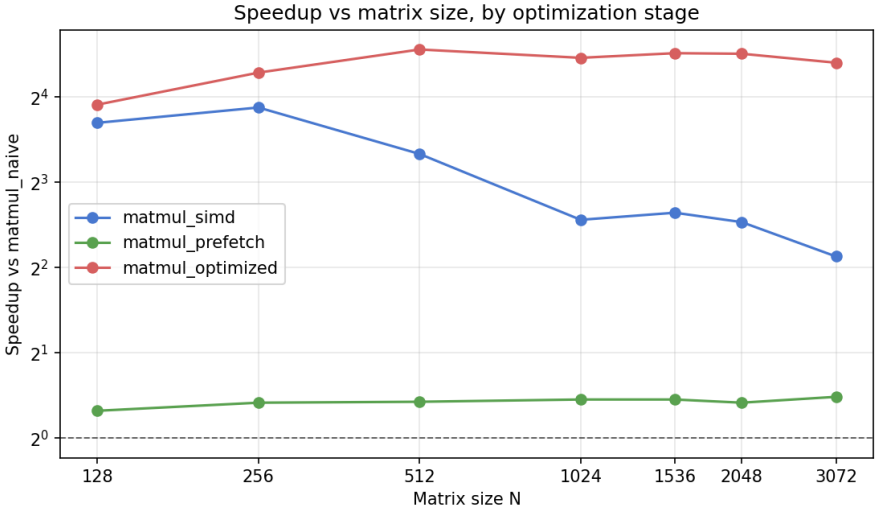

Figure 10: Speedup of `matmul_simd`, `matmul_prefetch`, and `matmul_optimized` over `matmul_naive` vs. matrix size (log-log).

Stage	Speedup @ 1024^3	Speedup @ 2048^3
`matmul_simd`	5.90×	5.79×
`matmul_prefetch`	1.37×	1.33×
`matmul_optimized`	23.81×	22.74×

Table 9: Speedup of each stage (median of 2 repeats).

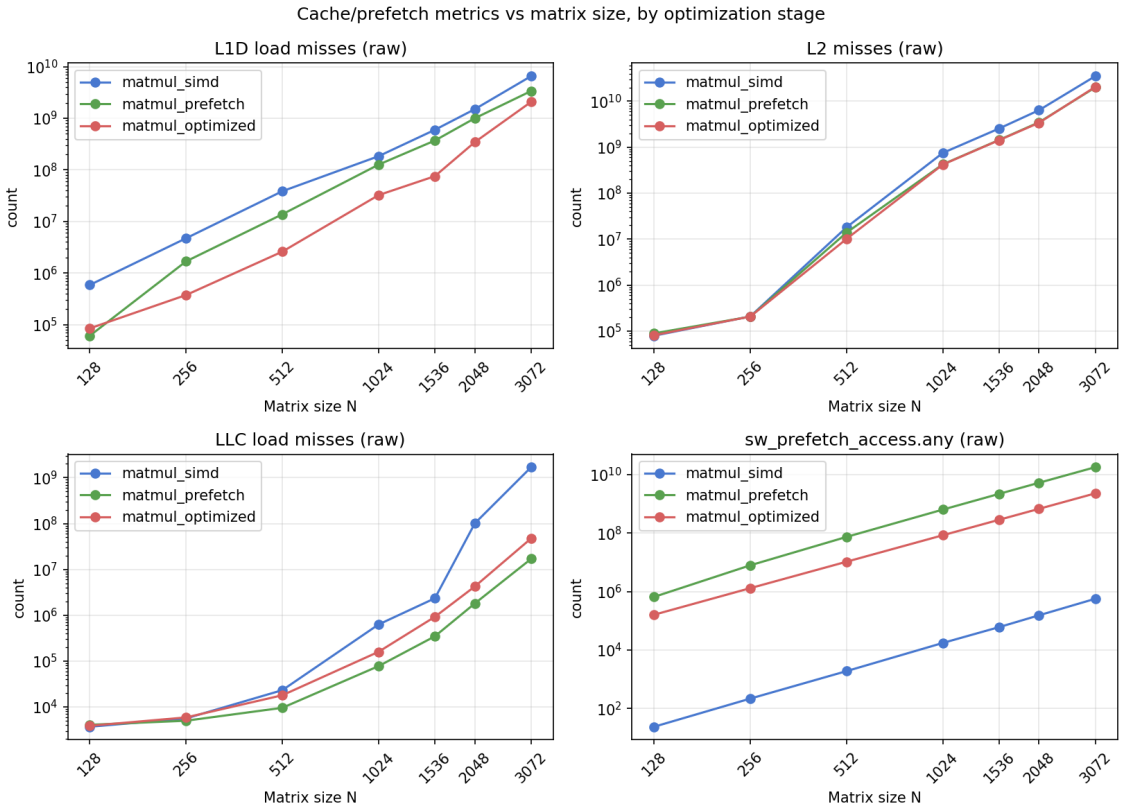

Figure 11: L1D / L2 / LLC miss counts and `sw_prefetch_access.any` vs. matrix size, one line per stage (raw `perf stat` counts, log-log).

3.3 Final Score

Full graded run (`./bin/matmul`, default N=1024, seed 1234):

```
stage           correct    time(ms)     GFLOP/s    speedup
-------------------------------------------------------------
naive (ref)     yes         518.481        4.14      1.00x
simd            yes          83.772       25.63      6.19x
prefetch        yes         376.951        5.70      1.38x
optimized       yes          22.284       96.37     23.27x
-------------------------------------------------------------
correctness : 30 / 30   (3 of 3 stages correct)
optimized   : 23.27x  ->  55 / 70  speedup points
TOTAL       : 85 / 100
```

3.4 Discussion

SIMD comes before tiling because the naive/scalar loop is compute-bound (a single dependent FMA chain per (i, j) over K): with nothing to overlap, cache blocking cannot show a benefit until the inner loop is fast enough to be memory-bound instead. This is visible in `matmul_prefetch` (scalar, tiled, prefetched): it fixes the memory side but stays compute-bound at scalar throughput, so its speedup is a flat $\sim 1.34\times$ regardless of size. Tiling and prefetch have nothing left to hide once the loop itself is the bottleneck.

`matmul_simd` shows the opposite failure: fast compute, but no tiling. Its speedup collapses from $\sim$ 13–15$\times$ at $N \leq 256$ (fits in L2/L3) to $\sim$ 4–6$\times$ at $N \geq 1024$, matching the cache-metrics plot:

L1D and LLC misses for `matmul_simd` grow far faster with N than for `matmul_optimized` or `matmul_prefetch` (e.g. at N=2048, LLC misses are 101M for `simd` vs. 4.2M for `optimized` and 1.8M for `prefetch`). With no tiling, B is re-streamed from memory on every row of A once it no longer fits in cache.

`matmul_optimized` combines both fixes and inherits the best of each: SIMD throughput plus `matmul_prefetch`-level cache behavior from the N-tiling, so its speedup stays flat at $\sim$ 22–24$\times$ for $N \geq 512$ instead of decaying like `matmul_simd`. Prefetch adds a further, smaller margin on top by hiding the residual DRAM latency on the 3 active B rows per internal loop iteration. `sw_prefetch_access.any` is essentially zero for `matmul_simd` (no explicit prefetch) and large for both `matmul_prefetch` and `matmul_optimized`, confirming it is this issued-prefetch traffic, not incidental hardware prefetching, driving the gap.

4 llama.cpp Results

Metric	llama.cpp native	Our implementation	Speedup
Prompt-eval (tok/s)	12,562.81	16,129.03	1.28×
Generation (tok/s)	7,437.14	9,826.86	1.32×

Table 10: `make llama-demo` comparison.